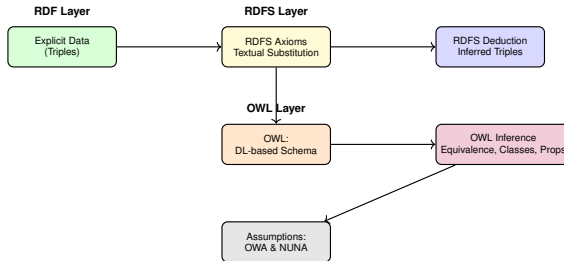


RDFS (Lecture 6)



Lecture 6



Lecture 6: From RDFS deduction to OWL reasoning (equivalence, constraints) under OWA (Open World Assumption) and NUNA (No Unique Name Assumption)

Agenda

■ 1. Textual Substitution

How RDFS applies schema rules to triples

■ 2. RDFS (continued from Lecture 5)

- Axiomatic System
- Deduction and inference of implicit knowledge

■ 3. Introduction to OWL

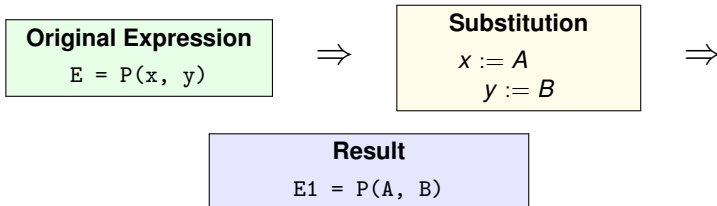
- Basic OWL Operators (classes, properties, restrictions)
- Logical Equality: `owl:sameAs` and `equivalentClass`
- Leibniz Inference Rule (Substitution of Identicals)
- OWL Axioms and their meaning
- Open World Assumption (OWA) & Non-Unique Name Assumption (NUNA)

From schema-driven inference in RDFS to expressive logical reasoning in OWL.

Textual Substitution

Textual substitution is a simple idea:

Variables are replaced in an Expression by new expressions.



Substitution = simultaneously replace all occurrences of variables by new expressions.

Textual Substitution [1] (1/4)

Definition

Let E and $E1$ be well-formed expressions of a formal language $\mathcal{L}$ (e.g., RDFS). Let X be a list of variables $\langle x_1, \dots, x_n \rangle$. Let R be a list of well-formed expressions $R_1, \dots, R_n$.

Textual Substitution [1] (1/4)

Definition

Let E and $E1$ be well-formed expressions of a formal language $\mathcal{L}$ (e.g., RDFS). Let X be a list of variables $\langle x_1, \dots, x_n \rangle$. Let R be a list of well-formed expressions $R_1, \dots, R_n$.

Notation

$E1$ is the textual substitution of X by R in E :

$$E[X := R] = E1$$

$E1$ is the result of simultaneously replacing in E all the instances of the variables x_i in X by the corresponding expression R_i in R .

Textual Substitution (2/4)

Abstract Example

Suppose X is a list of variables $\langle x_1, \dots, x_n \rangle$ and R is a list of well-formed expressions $R_1, \dots, R_n$.

In the textual substitution $E[X := R] = E1$

- x_1 is replaced by R_1
- x_2 is replaced by R_2
- ...
- x_n is replaced by R_n

Textual Substitution (2/4)

Abstract Example

Suppose X is a list of variables $\langle x_1, \dots, x_n \rangle$ and R is a list of well-formed expressions $R_1, \dots, R_n$.

In the textual substitution $E[X := R] = E1$

- x_1 is replaced by R_1
- x_2 is replaced by R_2
- ...
- x_n is replaced by R_n

Observation

$E1$ is called an instantiation of E .

Textual Substitution (3/4)

E: The Expression

IF ($P1$, $\text{rdfs:subPropertyOf}$, $P2$) **AND** ($P2$, $\text{rdfs:subPropertyOf}$, $P3$) **THEN** ($P1$, $\text{rdfs:subPropertyOf}$, $P3$)

Variables

- $P1$, $P2$, and $P3$ are variables.
- $X = \langle P1, P2, P3 \rangle$

R: The List of Expressions

- $R1 = \text{ex:coordinates}$
- $R2 = \text{ex:participates}$
- $R3 = \text{ex:takes_part}$

Textual Substitution (4/4)

Substitution

If $E[X := R] = E1$, then $E1$ is obtained by replacing:

- $P1$ with $R1 = \text{ex:coordinates}$
- $P2$ with $R2 = \text{ex:participates}$
- $P3$ with $R3 = \text{ex:takes_part}$

Textual Substitution (4/4)

Substitution

If $E[X := R] = E1$, then $E1$ is obtained by replacing:

- $P1$ with $R1 = \text{ex:coordinates}$
- $P2$ with $R2 = \text{ex:participates}$
- $P3$ with $R3 = \text{ex:takes_part}$

E1: The Resulting Expression

IF (ex:coordinates, rdfs:subPropertyOf, ex:participates)
AND (ex:participates, rdfs:subPropertyOf, ex:takes_part)
THEN (ex:coordinates, rdfs:subPropertyOf, ex:takes_part)

The diagram illustrates the process of generating RDFS axioms from a graph. It features two RDFS graphs, G1 and G2, connected by a central box labeled "AXIOMATIC SYSTEM RDFS". A curved arrow points from G1 to G2, labeled "Textual Substitutions Variables in RDFS axioms by resources, literal, or blank nodes in RDFS graph G1". Graph G1 is a small, simple graph, while Graph G2 is a large, complex graph with many nodes and edges.

Textual Substitution in RDFS

A2: Transitivity

```
IF (C1 subClassOf C2)
AND (C2 subClassOf C3)
THEN (C1 subClassOf C3)
```

Substitution

```
C1 := ex:French_King
C2 := ex:Ruler
C3 := ex:Person
```

Derived:

```
ex:French_King subClassOf
ex:Person
```

A3: Inheritance

```
IF (C1 subClassOf C2)
AND (X a C1)
THEN (X a C2)
```

Substitution

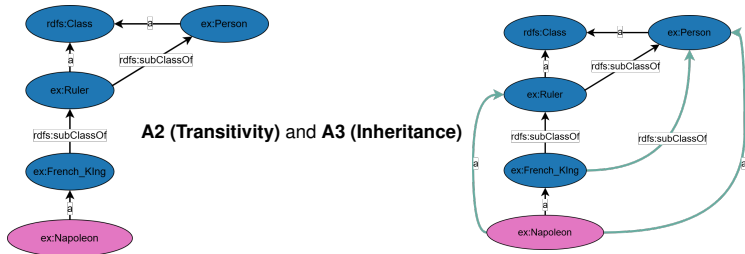
```
C1 := ex:French_King
C2 := ex:Ruler
X := ex:Napoleon
```

Derived:

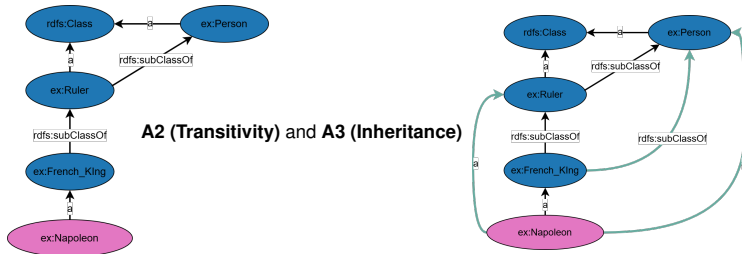
```
ex:Napoleon a ex:Ruler
```

RDFS inference = apply axioms after substitution → add the resulting triples.

Textual Substitution in RDFS (2/2)

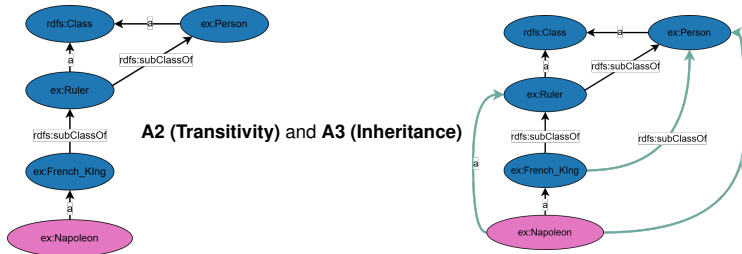


Textual Substitution in RDFS (2/2)



Textual Substitution $A2[[C1, C2, C3] := [ex:French_King, ex:Ruler, ex:Person]] = E1$
A2= IF $C1 \text{ rdfs:subClassOf } C2$ **AND** $C2 \text{ rdfs:subClassOf } C3$, **THEN** $C1 \text{ rdfs:subClassOf } C3$
 $C1$ is replaced by $ex:French_King$, $C2$ is replaced by $ex:Ruler$, $C3$ is replaced by $ex:Person$
E1: IF $ex:French_King \text{ rdfs:subClassOf } ex:Ruler$ **AND** $ex:Ruler \text{ rdfs:subClassOf } ex:Person$, **THEN** $ex:French_King \text{ rdfs:subClassOf } ex:Person$.

Textual Substitution in RDFS (2/2)



Textual Substitution A2[[C1, C2, C3] := [ex:French_King, ex:Ruler, ex:Person]] = E1
 A2= IF C1 rdfs:subClassOf C2 AND C2 rdfs:subClassOf C3, THEN C1 rdfs:subClassOf C3
 C1 is replaced by ex:French_King, C2 is replaced by ex:Ruler, C3 is replaced by ex:Person
 E1: IF ex:French_King rdfs:subClassOf ex:Ruler AND ex:Ruler rdfs:subClassOf ex:Person, THEN ex:French_King rdfs:subClassOf ex:Person.

Textual Substitution A3[[C1, C2, X] := [ex:French_King, ex:Ruler, ex:Napoleon]] = E1
 A3= IF C1 rdfs:subClassOf C2 AND X a C1, THEN X a C2
 C1 is replaced by ex:French_King, C2 is replaced by ex:Ruler, X is replaced by ex:Napoleon
 E1: IF ex:French_King rdfs:subClassOf ex:Ruler AND ex:Napoleon a ex:French_King, THEN ex:Napoleon a ex:Ruler.

Intuition: How Textual Substitution Works

Axiom Pattern (General Rule)

IF (C1 subClassOf C2) AND (C2 subClassOf C3)
THEN (C1 subClassOf C3)

⇓ **substitute variables** ⇓

Choose concrete replacements

C1 := D1 C2 := D2 C3 := D3

⇓

Instantiated Axiom (now a concrete rule)

IF (D1 subClassOf D2) AND (D2 subClassOf D3)
THEN (D1 subClassOf D3)

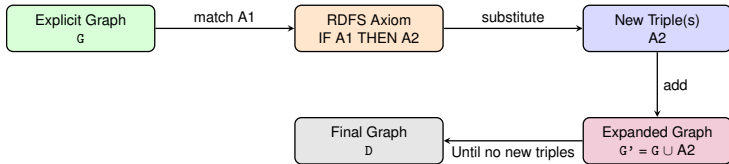
Textual substitution = “plug in” real terms into an axiom template.

Textual Substitution

Theorem

The result of a textual substitution in a theorem is also a theorem. If T is a theorem, THEN $T[X := R]$ is a theorem.

Intuition: RDFS as a Deduction System



RDFS repeatedly applies axioms to derive all implicit facts from the explicit graph.

RDFS as a Deduction System

Definition

Given two RDFS graphs G and D :

- D can be derived/entailed from G , i.e., $G \vdash D$, if and only if there is a sequence of RDFS graphs
- $G_1, \dots, G_k$ with $G_1 = G$ and $G_k = D$

RDFS as a Deduction System

Definition

Given two RDFS graphs G and D :

- D can be derived/entailed from G , i.e., $G \vdash D$, if and only if there is a sequence of RDFS graphs
- $G_1, \dots, G_k$ with $G_1 = G$ and $G_k = D$

For each j ($2 \leq j \leq k$), the following holds:

- There is an RDFS Axiom $\mathcal{A}$ of the form:

(IF $A1$ THEN $A2$), Z list of variables and R list of RDFS expressions

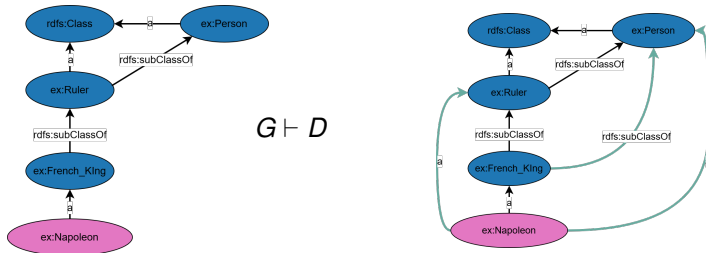
- Textual Substitution:

$(\text{IF } A1 \text{ THEN } A2)[Z := R] = (\text{IF } E1 \text{ THEN } E2)$

- Conditions:

- $E1 \subseteq G_{j-1}$
- $G_j = G_{j-1} \cup E2$

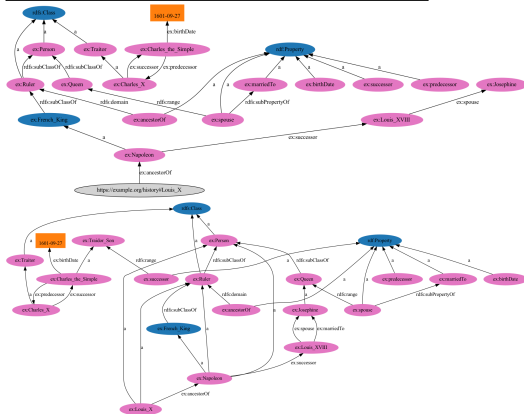
Textual Substitution in RDFS (2/2)



D is the result of applying axioms of group "A" to graph G

RDFS Deduction System

G is the RDFS graph of Napoleon



$G \vdash D$

D is the result of applying RDFS axioms to graph G (Lecture 5)

Inference in RDFS: Original and New Triples

Part of the Original RDFS Triples:

ex:Napoleon a ex:French_King ;
ex:successor ex:Louis_XVIII.
ex:Louis_XVIII ex:marriedTo ex:Josephine .

New Triples Added After Inference:

ex:Napoleon a ex:Ruler .
ex:Napoleon a ex:Person .
ex:successor rdfs:range ex:Traitor_Son .
ex:Louis_X a ex:Ruler .
ex:Louis_XVIII ex:marriedTo ex:Josephine .

Inference in RDFS: Original and New Triples

Part of the Original RDFS Triples:

ex:Napoleon a ex:French_King ;
ex:successor ex:Louis_XVIII.
ex:Louis_XVIII ex:marriedTo ex:Josephine .

New Triples Added After Inference:

ex:Napoleon a ex:Ruler .
ex:Napoleon a ex:Person .
ex:successor rdfs:range ex:Traitor_Son .
ex:Louis_X a ex:Ruler .
ex:Louis_XVIII ex:marriedTo ex:Josephine .

Key Points:

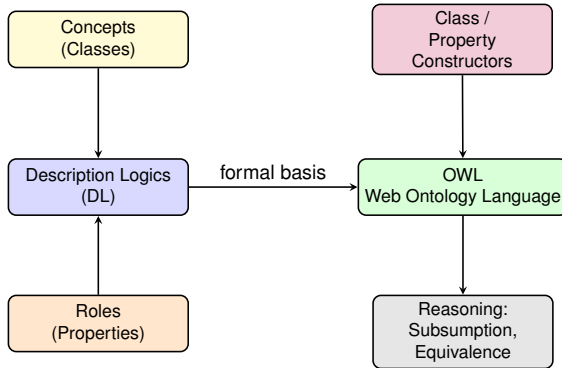
When RDFS inference is applied, additional triples are generated based on schema definitions such as:

- `rdfs:subClassOf` (class hierarchies)
- `rdfs:domain` and `rdfs:range` (property constraints)

Summary RDFS

RDFS Positive Points	RDFS Negative Points
Data model to represent RDF schemas in terms of classes, properties (object and data type properties, and domain and range)	Negation cannot be represented
Open World Assumption	Cardinality constraints cannot be represented
RDFS Axioms are used to deduce missing relationships	- Set of classes cannot be specified - Domain integrity constraints cannot be specified - Object Properties cannot have attributes

Intuition: OWL and Description Logics



DL provides the logical foundation; OWL adds constructors, vocabulary, and machine reasoning support.

OWL and Description Logics

Description Logics

Description Logics (DL):

- A restricted first-order language.
- Allows for the representation of knowledge in terms of:
 - Terminological (abstract) concepts
 - Instances or individuals of abstract concepts
 - Subsumption relationships
 - Equivalence
 - Class construction
 - Property construction

Description Logics (DL)

Concepts

- ****Classes****: Unary predicates.
- ****Roles****: Represent binary relations between instances of classes.
- ****Individuals****: Instances of a class.

Limited Expressiveness

- Reduced set of constructors to build concepts and complex roles.

Classification of Concepts and Individuals

- Subconcepts/Superconcepts

Description Logics (DL)

A Theory in DL

A theory (knowledge base) in DL is composed of:

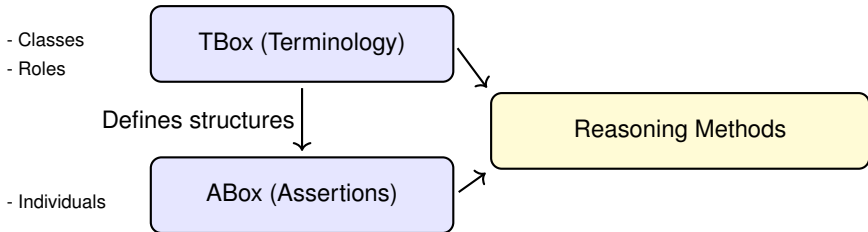
- TBox: Terminology or vocabulary
 - Classes/Concepts: Denote sets of individuals.
 - Roles: Binary relations between individuals.
- ABox: Set of statements about individuals.

Description Logics (DL)

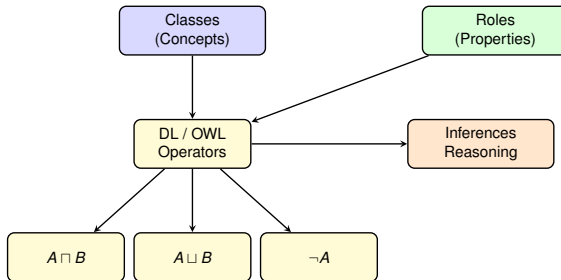
A Theory in DL

A theory (knowledge base) in DL is composed of:

- TBox: Terminology or vocabulary
 - Classes/Concepts: Denote sets of individuals.
 - Roles: Binary relations between individuals.
- ABox: Set of statements about individuals.



Intuition: OWL / Description Logic Operators



DL/OWL operators let us build complex classes, define constraints, and enable automated logical reasoning.

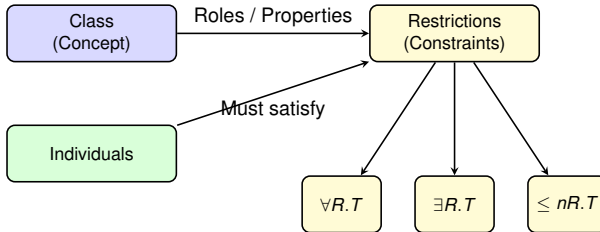
DL Operators (1/2)

Operator	Description	Example
A	Atomic Class	PERSON
$\top$	Thing-Universal Concept	
$\perp$	Empty Concept	
$A \sqcap B$	Intersection of Classes	STUDENT $\sqcap$ FEMALE
$A \sqcup B$	Union of Classes	MasterStudent $\sqcup$ PhDStudent
$\neg A$	Complement of a Class	$\neg$ FEMALE
$A \sqsubseteq B$	SubClass	FemaleStudent $\sqsubseteq$ STUDENT
$A \equiv B$	Equivalent Classes	MEN $\equiv$ MaleHuman
$\{x_1, x_2, \dots, x_n\}$	Enumeration of Instances	{CS, Math, Physics, Chemistry}

DL Operators (2/2)

Operator	Description	Example
$P_1 \sqsubseteq P_2$	SubProperty	<code>ex:coordinates</code> $\sqsubseteq$ <code>ex:participates</code>
$\exists R \sqsubseteq A$	A is the domain of R	$\exists$ <code>ex:coordinates</code> $\sqsubseteq$ <code>ex:Senior_Researcher_MD</code>
$\exists R^- \sqsubseteq B$	B is the range of R	$\exists$ <code>ex:coordinates</code> ⁻ $\sqsubseteq$ <code>ex:Research_project</code>
$A \sqcap B \sqsubseteq \perp$	A disjoint with B	<code>MALE</code> $\sqcap$ <code>FEMALE</code> $\sqsubseteq \perp$
$A(x)$	x is an instance of A	<code>ex:JohnSmith</code> is a <code>ex:Researcher</code>
$R(x, y)$	R relates x with y	<code>ex:JohnSmith</code> <code>ex:coordinates</code> <code>ex:CLARIFY</code>
$x \equiv y$	x is equivalent to y	<code>ex:CLARIFY</code> <code>owl:sameAs</code> <code>ex:EUCLARIFY</code>

Intuition: DL Integrity Constraints



DL integrity constraints restrict **how many** and **what kind** of related individuals a class may have.

They enable OWL reasoners to detect inconsistencies and derive implicit knowledge.

DL Integrity Constraints

Operator	Description	Example
$\forall R.T$	Instances in R related to only entities in T	$\forall \text{ex:takes_part.Researcher_MD}$
$\exists R.T$	Instances in R related to at least one entity in T	$\exists \text{ex:takes_part.Senior_Researcher_MD}$
$\exists R.\{x_1\}$	Existential quantifier in an enumerated set	$\exists \quad \text{ex:coordinates}\{ \text{ex:JohnSmith}, \text{ex:MarySmith} \}$
$\geq nR.T$	Instances in R related to at least n entities in T	$\geq 2 \text{ex:coordinates.Research_project}$
$\leq nR.T$	Instances in R related to at most n entities in T	$\leq 2 \text{ex:coordinates.Research_project}$
$= nR.T$	Instances in R related to exactly n entities in T	$= 2 \text{ex:coordinates.Research_project}$

What is OWL?

Web Ontology Language (OWL)

- Part of the W3C RDF recommendation family.
- Based on Description Logics
- Extends RDFS and is able to express further schema definitions in RDF, e.g.,
 - Equality of individuals (`owl:sameAs`)
 - Equivalence or disjointness of properties (`owl:equivalentProperty`, `owl:propertyDisjointWith`)
 - Equivalence or disjointness of classes (`owl:equivalentClass`, `owl:disjointWith`)
 - Definitions of complex classes

OWL as a W3C Standard

OWL

Page Discussion

Read View source View history Tools ▾

Web Ontology Language (OWL)



- Publication date: 2012-12-11 (with a previous version published at: 2009-10-27)
- Created by: [OWL Working Group](#)
- List of documents at: <http://www.w3.org/standards/techs/owl>

Overview

The W3C Web Ontology Language (OWL) is a Semantic Web language designed to represent rich and complex knowledge about things, groups of things, and relations between things. OWL is a computational logic-based language such that knowledge expressed in OWL can be exploited by computer programs, e.g., to verify the consistency of that knowledge or to make implicit knowledge explicit. OWL documents, known as ontologies, can be published in the World Wide Web and may refer to or be referred from other OWL ontologies. OWL is part of the W3C's Semantic Web technology stack, which includes [RDF](#), [RDFS](#), [SPARQL](#), etc.

The current version of OWL, also referred to as "OWL 2", was developed by the [\[W3C OWL Working Group\]](#) (now closed) and published in 2009, with a Second Edition published in 2012. OWL 2 is an extension and revision of the 2004 version of OWL developed by the [\[W3C Web Ontology Working Group\]](#) (now closed) and published in 2004. The deliverables that make up the OWL 2 specification include a [Document Overview](#), which serves as an introduction to OWL 2, describes the relationship between OWL 1 and OWL 2, and provides an entry point to the remaining deliverables via a [Documentation Roadmap](#).

Recommended Reading

As can be seen from the above mentioned [Documentation Roadmap](#), OWL 2 is normatively defined by five core specification documents describing its conceptual structure, primary exchange syntax (RDF/XML), two alternative semantics (Direct and RDF-Based), and conformance requirements. Three additional specification documents describe optional features that may be supported by some implementations: the language profiles, and two alternative concrete syntaxes (OWL/XML and Manchester).

These documents are, however, all rather technical and mainly aimed at OWL 2 implementers and tool developers. Those looking for a more approachable guide to the features and usage of OWL 2 may prefer to consult one of the user documents, which include a [Primer](#) and a [Quick Reference Guide](#).

A number of textbooks have been published on OWL, and on Semantic Web in general. Please, refer to a [separate page](#) listing some of those, as maintained by the community. That list also includes references to conference proceedings and article collections that might be of general interest.

Tools that are listed as relevant to OWL

(Note that you can browse tools per [tool categories](#) or [programming languages](#), too.)

1

¹<https://www.w3.org/OWL/>

Benefits of OWL as a W3C Standard

Benefits of OWL

- **Interoperability:** OWL facilitates seamless data sharing and integration across systems and organizations due to its standardized structure.
- **Global Adoption:** As a W3C standard, OWL is widely recognized and adopted by governments, industries, and academia worldwide.
- **Compatibility:** OWL builds on existing W3C standards such as RDF and RDFS, ensuring compatibility with other semantic web technologies.
- **Extensibility:** OWL provides a flexible framework for defining domain-specific ontologies, supporting the evolving needs of different applications.
- **Reasoning Support:** OWL enables logical reasoning to deduce new knowledge, validate data, and ensure consistency.
- **Community and Tooling:** As a W3C standard, OWL benefits from a robust ecosystem of tools, libraries, and an active global community.
- **Trust and Reliability:** Being a W3C standard, OWL ensures a reliable, well-documented, and long-term supported technology.

Intuition: What Does OWL Add?

OWL introduces three basic kinds of building blocks that make our knowledge more expressive than with RDFS alone.

1. **owl:Class** — What kinds of things exist?

Examples: `ex:Person`, `ex:Course`
OWL uses **classes** to group individuals.

2. **owl:ObjectProperty** — How do things relate?

Examples: `ex:hasParent`, `ex:teaches`
Describes connections *between* individuals.

3. **owl:DataProperty** — What data do things have?

Examples: `ex:hasAge`, `ex:hasName`
Links an individual to a literal value.

OWL — a richer vocabulary for describing classes, relations, and data

Operators of OWL: Prefix and New Predicates (1/2)

@prefix owl: <<http://www.w3.org/2002/07/owl#>>

New Predicates in OWL

- **owl:Class:** Defines a class or a set of individuals sharing common characteristics.
 - Example: `owl:Class ex:Person`
 - Usage: Individuals such as `ex:John` and `ex:Mary` are members of `ex:Person`.
- **owl:ObjectProperty:** Represents a property between two individuals.
 - Example: `owl:ObjectProperty ex:hasParent`
 - Usage: `ex:John ex:hasParent ex:Mary`
- **owl:DataProperty:** Represents a property linking an individual to a literal value.
 - Example: `owl:DataProperty ex:hasAge`
 - Usage: `ex:John ex:hasAge "30"`

Intuition: Logical Equivalence in OWL

Logical equivalence means “these represent the same thing”.

OWL provides three ways to express this equivalence.

Three Forms of Equivalence

- **Equivalent Classes:** `A owl:equivalentClass B`
- **Equivalent Properties:** `P1 owl:equivalentProperty P2`
- **Equivalent Individuals:** `a owl:sameAs b`



All of them refer to the same concept or object.

Logical Properties

- **Symmetry:** If A is equivalent to B, then B is equivalent to A.
- **Transitivity:** If A is equivalent to B, and B to C, then A is equivalent to C.

Logical Equivalence in OWL

Logical Equivalence ($\equiv$) in OWL

Three properties represent logical equivalence:

- **Equivalent Classes:** `A owl:equivalentClass B`
- **Equivalent Properties:** `P1 owl:equivalentProperty P2`
- **Equivalent Individuals:** `a owl:sameAs b`

Properties of the Logical Equivalence

- **Symmetry:** If $A \equiv B$, then $B \equiv A$.
- **Transitivity:** If $A \equiv B$ and $B \equiv C$, then $A \equiv C$.

Examples of Logical Equivalence in OWL

Examples

■ Equivalent Classes:

- `ex:Human owl:equivalentClass ex:HomoSapiens`
- Individuals of `ex:Human` are also members of `ex:HomoSapiens`.

■ Equivalent Properties:

- `ex:hasParent owl:equivalentProperty ex:hasProgenitor`
- Relationship `hasParent(x, y)` is logically the same as `hasProgenitor(y, x)`.

■ Equivalent Individuals:

- `ex:John owl:sameAs ex:Jonathan`
- `ex:John` and `ex:Jonathan` are the same entity.

Examples in OWL

`ex:humanFiona owl:sameAs ex:ogreFiona`



Human Fiona

```
ex:humanFiona a          ex:HumanFictionalCharacter;  
                  ex:chosenWifeBy ex:LordFarquaad;  
                  ex:isFatherOf   ex:KingHarold;  
                  ex:isMotherOf   ex:QueenLillian.
```

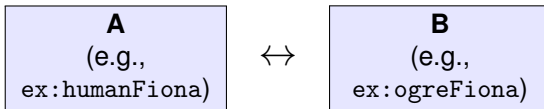


Ogre Fiona

```
ex:ogreFiona a          ex:OgreFictionalCharacter;  
                  ex:loves      ex:Shrek;  
                  ex:isOgreMomOf ex:Felicia;  
                  ex:isOgreMomOf ex:Fergus;  
                  ex:isOgreMomOf ex:Farkle.
```

Intuition: The Leibniz Inference Rule

If two things are **equivalent**, then one can be **substituted** for the other *in any expression* without changing its meaning.



A and B are declared equivalent (owl:sameAs)

Intuition: Equivalence means “same meaning,” so substituting one for the other *cannot change the truth* of the expression.

Inference Rule based on Equivalence (1/2)

Leibniz Inference Rule

- The rule states: **If $A \equiv B$, then replacing A with B in any well-formed expression E preserves equivalence.**
- Formal Definition:

$$\text{If } A \equiv B, \text{ then } E[X := A] \equiv E[X := B]$$

Inference Rule Based on Equivalence (2/2)

`ex:humanFiona owl:sameAs ex:ogreFiona`

Graph E:

```
X a                ex:HumanFictionalCharacter;  
  ex:chosenWifeBy  ex:LordFarquaad;  
  ex:isFatherOf    ex:KingHarold;  
  ex:isMotherOf    ex:QueenLillian.
```

Inference Rule Based on Equivalence (2/2)

`ex:humanFiona owl:sameAs ex:ogreFiona`

Graph E:

```
X a                ex:HumanFictionalCharacter;  
  ex:chosenWifeBy ex:LordFarquaad;  
  ex:isFatherOf   ex:KingHarold;  
  ex:isMotherOf   ex:QueenLillian.
```

Graph G: $E[X:=ex:humanFiona]$:

```
ex:humanFiona a      ex:HumanFictionalCharacter;  
  ex:chosenWifeBy ex:LordFarquaad;  
  ex:isFatherOf   ex:KingHarold;  
  ex:isMotherOf   ex:QueenLillian.
```

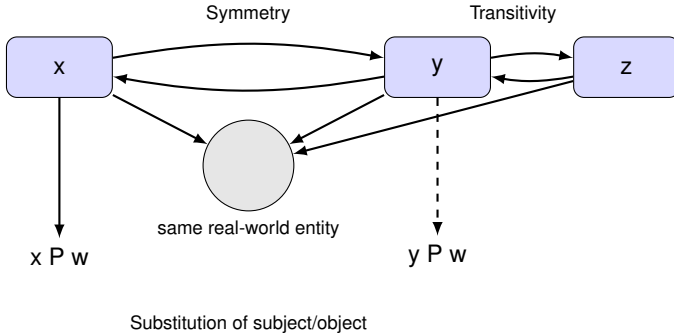
Graph H: $E[X:=ex:ogreFiona]$:

```
ex:ogreFiona a       ex:HumanFictionalCharacter;  
  ex:chosenWifeBy ex:LordFarquaad;  
  ex:isFatherOf   ex:KingHarold;  
  ex:isMotherOf   ex:QueenLillian.
```

Conclusion:

- Since `ex:humanFiona owl:sameAs ex:ogreFiona`, replacing `humanFiona` with `ogreFiona` preserves equivalence and $\text{Graph G} \equiv \text{Graph H}$

Intuition: owl:sameAs Axioms



OWL Axioms

Group E (SameAs)

1. **Symmetry:** IF $(x, \text{owl:sameAs}, y)$
THEN $(y, \text{owl:sameAs}, x)$
2. **Transitivity:** IF $(x, \text{owl:sameAs}, y)$ AND $(y, \text{owl:sameAs}, z)$
THEN $(x, \text{owl:sameAs}, z)$
3. **Equivalent Subject:** IF $(x, \text{owl:sameAs}, y)$ AND (x, P, z)
THEN (y, P, z)
4. **Equivalent Object:** IF $(x, \text{owl:sameAs}, y)$ AND (z, P, x)
THEN (z, P, y)

OWL SameAs

Given Logical Equivalence

- `ex:humanFiona owl:sameAs ex:ogreFiona`
- This means `ex:humanFiona` and `ex:ogreFiona` are logically equivalent individuals.

Symmetry of owl:sameAs

Symmetry Axiom:

- **Axiom E1:** If x owl:sameAs y , then y owl:sameAs x .
- **Explanation:** If one individual is equivalent to another, the reverse is also true.
- **Example:**
ex:humanFiona owl:sameAs ex:ogreFiona
implies
ex:ogreFiona owl:sameAs ex:humanFiona.

Symmetry of owl:sameAs

Symmetry Axiom:

- **Axiom E1:** If x owl:sameAs y , then y owl:sameAs x .
- **Explanation:** If one individual is equivalent to another, the reverse is also true.
- **Example:**
ex:humanFiona owl:sameAs ex:ogreFiona
implies
ex:ogreFiona owl:sameAs ex:humanFiona.

Conclusion: The symmetry axiom ensures that equivalence relationships are bidirectional, allowing consistent inference when either individual is referenced in an equivalence statement.

E3. Equivalent Subject Axiom (1/2)

OWL Graph G1

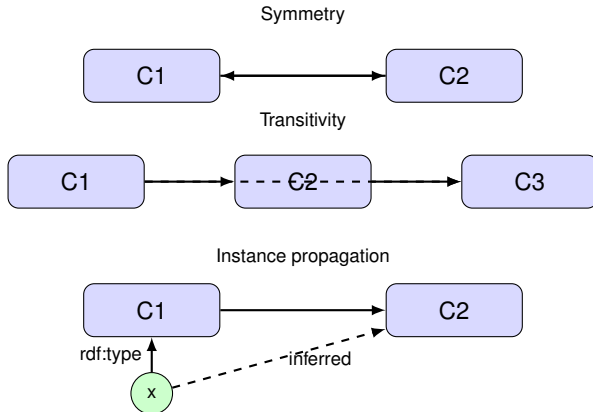
```
ex:humanFiona a          ex:HumanFictionalCharacter;  
                  ex:chosenWifeBy ex:LordFarquaad;  
                  ex:isFatherOf   ex:KingHarold;  
                  ex:isMotherOf   ex:QueenLillian.  
ex:ogreFiona a          ex:OgreFictionalCharacter;  
                  ex:loves        ex:Shrek;  
                  ex:isOgreMomOf  ex:Felicia;  
                  ex:isOgreMomOf  ex:Fergus;  
                  ex:isOgreMomOf  ex:Farkle.
```

Deduced Graph G2 based on Axiom E3

```
ex:ogreFiona a          ex:HumanFictionalCharacter;  
                  ex:chosenWifeBy ex:LordFarquaad;  
                  ex:isFatherOf   ex:KingHarold;  
                  ex:isMotherOf   ex:QueenLillian.  
ex:humanFiona a          ex:OgreFictionalCharacter;  
                  ex:loves        ex:Shrek;  
                  ex:isOgreMomOf  ex:Felicia;  
                  ex:isOgreMomOf  ex:Fergus;  
                  ex:isOgreMomOf  ex:Farkle.
```

UNION G1 (i.e., all the triples from G1)

Intuition: owl:equivalentClass



Equivalent classes behave like identical categories: symmetry, transitivity, and shared instances.

OWL Axioms

Group F (Equivalent Classes)

1. Symmetry: IF (C1, owl:equivalentClass, C2)
THEN (C2, owl:equivalentClass, C1)

2. Transitivity: IF (C1, owl:equivalentClass, C2)
AND (C2, owl:equivalentClass, C3)
THEN (C1, owl:equivalentClass, C3)

3. Equivalent Class Instances:
IF (C1, owl:equivalentClass, C2)
AND (x, rdf:type, C1)
THEN (x, rdf:type, C2)

OWL Axioms: Group F (Equivalent Classes)

F1. Symmetry Example:

- **Axiom:** If $(C1, \text{owl:equivalentClass}, C2)$, then $(C2, \text{owl:equivalentClass}, C1)$.
- **Example:**
`ex:OgreMom owl:equivalentClass ex:MotherOfOgre.`
- **Inferred:** `ex:MotherOfOgre owl:equivalentClass ex:OgreMom.`

OWL Axioms: Group F (Equivalent Classes)

F1. Symmetry Example:

- **Axiom:** If $(C1, \text{owl:equivalentClass}, C2)$, then $(C2, \text{owl:equivalentClass}, C1)$.
- **Example:**
`ex:OgreMom owl:equivalentClass ex:MotherOfOgre.`
- **Inferred:** `ex:MotherOfOgre owl:equivalentClass ex:OgreMom.`

Conclusion: The symmetry axiom ensures that equivalence relationships between classes are bidirectional, allowing for logical consistency when reasoning about class hierarchies or shared properties.

OWL Axioms: Group F (Equivalent Classes)

F2. Transitivity Example:

- **Axiom:** If (C1, owl:equivalentClass, C2) and (C2, owl:equivalentClass, C3), then (C1, owl:equivalentClass, C3).
- **Example:**
ex:OgreMom owl:equivalentClass ex:MotherOfOgre.
ex:MotherOfOgre owl:equivalentClass ex:MaternalFigure.
- **Inferred:**
ex:OgreMom owl:equivalentClass ex:MaternalFigure.

OWL Axioms: Group F (Equivalent Classes)

F2. Transitivity Example:

- **Axiom:** If (C1, owl:equivalentClass, C2) and (C2, owl:equivalentClass, C3), then (C1, owl:equivalentClass, C3).
- **Example:**
ex:OgreMom owl:equivalentClass ex:MotherOfOgre.
ex:MotherOfOgre owl:equivalentClass ex:MaternalFigure.
- **Inferred:**
ex:OgreMom owl:equivalentClass ex:MaternalFigure.

Conclusion: The transitivity axiom ensures that equivalences propagate across multiple relationships, enabling reasoning over indirect equivalence chains to maintain logical consistency.

OWL Axioms: Group F (Equivalent Classes)

F3. Equivalent Class Instances Example:

- **Axiom:** If $(C1, \text{owl:equivalentClass}, C2)$ and $(x, \text{rdf:type}, C1)$, then $(x, \text{rdf:type}, C2)$.
- **Example:**
`ex:OgreMom owl:equivalentClass ex:MotherOfOgre.`
`ex:Fiona rdf:type ex:OgreMom.`
- **Inferred:**
`ex:Fiona rdf:type ex:MotherOfOgre.`

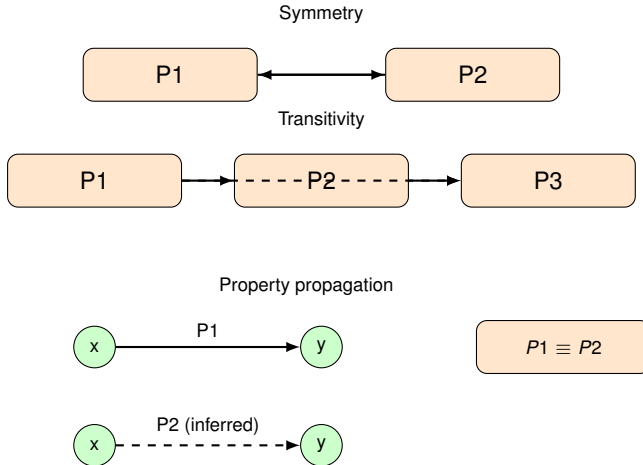
OWL Axioms: Group F (Equivalent Classes)

F3. Equivalent Class Instances Example:

- **Axiom:** If (C1, owl:equivalentClass, C2) and (x, rdf:type, C1), then (x, rdf:type, C2).
- **Example:**
ex:OgreMom owl:equivalentClass ex:MotherOfOgre.
ex:Fiona rdf:type ex:OgreMom.
- **Inferred:**
ex:Fiona rdf:type ex:MotherOfOgre.

Conclusion: The equivalent class instances axiom ensures that individuals belonging to one class are automatically inferred to belong to its equivalent class, maintaining logical consistency and simplifying reasoning about class membership.

Intuition: owl:equivalentProperty



OWL Axioms

Group G (Equivalent Properties)

1. Symmetry: IF (P1, owl:equivalentProperty, P2)
THEN (P2, owl:equivalentProperty, P1)

2. Transitivity: IF (C1, owl:equivalentProperty, C2)
AND (C2, owl:equivalentProperty, C3)
THEN (C1, owl:equivalentProperty, C3)

3. Equivalent Property Instances:
IF (P1, owl:equivalentProperty, P2)
AND (x, P1, y)
THEN (x, P2, y)

OWL Axioms: Group G (Equivalent Properties)

1. Symmetry:

- **Axiom:** IF (P1, owl:equivalentProperty, P2), **THEN** (P2, owl:equivalentProperty, P1).
- **Example:**
ex:isMother owl:equivalentProperty ex:motherOf.
- **Inferred:**
ex:motherOf owl:equivalentProperty ex:isMother.

OWL Axioms: Group G (Equivalent Properties)

1. Symmetry:

- **Axiom:** IF (P1, owl:equivalentProperty, P2), THEN (P2, owl:equivalentProperty, P1).
- **Example:**
ex:isMother owl:equivalentProperty ex:motherOf.
- **Inferred:**
ex:motherOf owl:equivalentProperty ex:isMother.

Conclusion: Symmetry ensures that equivalent properties can be used interchangeably, supporting bidirectional logical consistency in property relationships.

OWL Axioms: Equivalent Properties - Transitivity

Transitivity:

- **Axiom:** IF (P1, owl:equivalentProperty, P2) **AND** (P2, owl:equivalentProperty, P3), **THEN** (P1, owl:equivalentProperty, P3).
- **Example:**
ex:isParentOf owl:equivalentProperty ex:isAncestorOf.
ex:isAncestorOf owl:equivalentProperty
ex:isFamilyMemberOf.
- **Inferred:**
ex:isParentOf owl:equivalentProperty ex:isFamilyMemberOf.

OWL Axioms: Equivalent Properties - Transitivity

Transitivity:

- **Axiom:** IF (P1, owl:equivalentProperty, P2) **AND** (P2, owl:equivalentProperty, P3), **THEN** (P1, owl:equivalentProperty, P3).
- **Example:**
ex:isParentOf owl:equivalentProperty ex:isAncestorOf.
ex:isAncestorOf owl:equivalentProperty
ex:isFamilyMemberOf.
- **Inferred:**
ex:isParentOf owl:equivalentProperty ex:isFamilyMemberOf.

Conclusion: Property equivalences propagate transitively, creating a chain of equivalence relationships.

OWL Axioms: Equivalent Properties - Instances

Equivalent Property Instances:

- **Axiom:** IF (P1, owl:equivalentProperty, P2) **AND** (x, P1, y),
THEN (x, P2, y).
- **Example:**
ex:isParentOf owl:equivalentProperty ex:isAncestorOf.
ex:John ex:isParentOf ex:Mary.
- **Inferred:**
ex:John ex:isAncestorOf ex:Mary.

OWL Axioms: Equivalent Properties - Instances

Equivalent Property Instances:

- **Axiom:** IF (P1, owl:equivalentProperty, P2) **AND** (x, P1, y),
THEN (x, P2, y).
- **Example:**
ex:isParentOf owl:equivalentProperty ex:isAncestorOf.
ex:John ex:isParentOf ex:Mary.
- **Inferred:**
ex:John ex:isAncestorOf ex:Mary.

Conclusion: Equivalent properties ensure that relationships between entities are logically consistent and interchangeable across equivalent properties.

Examples in OWL



ex:ogreFiona

ex:ogreFiona a ex:OgreMom;
ex:isOgreMomOf ex:Felicia, ex:Fergus,
ex:Farkle.



ex:Felicia, ex:Fergus, ex:Farkle

ex:OgreMom owl:equivalentClass ex:MotherOfOgre.
ex:isOgreMomOf owl:equivalentProperty ex:isMotherOf.

ex:ogreFiona a ex:MotherOfOgre.
ex:ogreFiona ex:isMotherOf ex:Felicia, ex:Fergus, ex:Farkle.

Axiom 1: Equivalent Classes

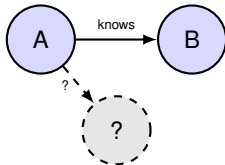
If $C1$ owl:equivalentClass $C2$, then the set of instances of $C1$ is the same as the set of instances of $C2$.

Axiom 2: Equivalent Properties

If $P1$ owl:equivalentProperty $P2$, then the set of instances of $P1$ is the same as the set of instances of $P2$.

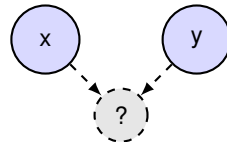
Intuition: OWL Assumptions (OWA & NUNA)

Open World Assumption



Unknown $\neq$ False

Non-Unique Name Assumption



x and y may be the same entity

OWA: Lack of information does not imply falsehood. **NUNA:** IRIs may refer to the same thing.

Assumptions in OWL

Open World Assumption (OWA)

The Open World Assumption (OWA) assumes that what is not known to be true is simply unknown, not false.

Assumptions in OWL

Open World Assumption (OWA)

The Open World Assumption (OWA) assumes that what is not known to be true is simply unknown, not false.

Non-Unique Name Assumption (NUNA)

Two or more IRIs may refer to the same thing! Without further information, two or more resources can correspond to the same.

Assumptions in OWL

Open World Assumption (OWA)

The Open World Assumption (OWA) assumes that what is not known to be true is simply unknown, not false.

Non-Unique Name Assumption (NUNA)

Two or more IRIs may refer to the same thing! Without further information, two or more resources can correspond to the same.

Why are these assumptions important:

- OWA supports reasoning with incomplete data and is well-suited for distributed knowledge sources like the web.
- NUNA aligns with the flexibility required for data integration and ontology alignment.

Example of OWA and NUNA (1/2)



ex:ogreFiona

ex:ogreFiona a ex:OgreMom;
ex:isOgreMomOf ex:Felicia, ex:Fergus,
ex:Farkle.



ex:Felicia, ex:Fergus, ex:Farkle

ex:OgreMom owl:equivalentClass ex:MotherOfOgre.
ex:isOgreMomOf owl:equivalentProperty ex:isMotherOf.

ex:ogreFiona a ex:MotherOfOgre.
ex:ogreFiona ex:isMotherOf ex:Felicia, ex:Fergus, ex:Farkle.

Question: How many kids does ex:ogreFiona have?

- (a) ex:ogreFiona has three kids.
- (b) ex:ogreFiona has at least three kids.
- (c) ex:ogreFiona has at least one kid.

Example of OWA and NUNA (2/2)

Answer: (c) `ex:ogreFiona` has at least one kid.

Explanation:

- **Open World Assumption (OWA):** The OWL graph states that `ex:ogreFiona` is related to `ex:Felicia`, `ex:Fergus`, and `ex:Farkle`, but it is unknown if she has additional children.
- Without explicit negation, OWL cannot infer that there are exactly three children; it only guarantees that there is **at least one**.
- **Non-Unique Name Assumption (NUNA):** Even though the names `ex:Felicia`, `ex:Fergus`, and `ex:Farkle` are listed, they could refer to the same individual unless explicitly stated otherwise.

Example: OWL Axioms



- Demonstrates OWL properties
- Will use this to explore OWL Axioms



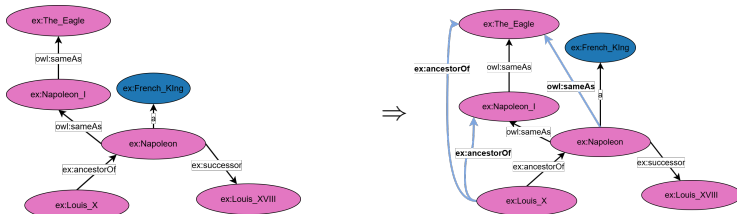
Example OWL graph: French Emperor
Napoleon Bonaparte

Exercise

Participate in the survey at Stud.IP.

OWL Axioms Summary		
Group E: SameAs		
Identifier	Title	Description
E1	Symmetry	If $x \text{ owl:sameAs } y$, then $y \text{ owl:sameAs } x$.
E2	Transitivity	If $x \text{ owl:sameAs } y$ and $y \text{ owl:sameAs } z$, then $x \text{ owl:sameAs } z$.
E3	Equivalent Subject	If $x \text{ owl:sameAs } y$ and x, P, z , then y, P, z .
E4	Equivalent Object	If $x \text{ owl:sameAs } y$ and z, P, x , then z, P, y .
Group F: Equivalent Classes		
F1	Symmetry	If $C1 \text{ owl:equivalentClass } C2$, then $C2 \text{ owl:equivalentClass } C1$.
F2	Transitivity	If $C1 \text{ owl:equivalentClass } C2$ and $C2 \text{ owl:equivalentClass } C3$, then $C1 \text{ owl:equivalentClass } C3$.
F3	Equivalent Class Instances	If $C1 \text{ owl:equivalentClass } C2$ and $x \text{ rdf:type } C1$, then $x \text{ rdf:type } C2$.
Group G: Equivalent Properties		
G1	Symmetry	If $P1 \text{ owl:equivalentProperty } P2$, then $P2 \text{ owl:equivalentProperty } P1$.
G2	Transitivity	If $P1 \text{ owl:equivalentProperty } P2$ and $P2 \text{ owl:equivalentProperty } P3$, then $P1 \text{ owl:equivalentProperty } P3$.
G3	Equivalent Property Instances	If $P1 \text{ owl:equivalentProperty } P2$ and $x, P1, y$, then $x, P2, y$.

Example: OWL Axioms Group E (1/2)



Axiom:

■ E2 (Transitivity) Example:

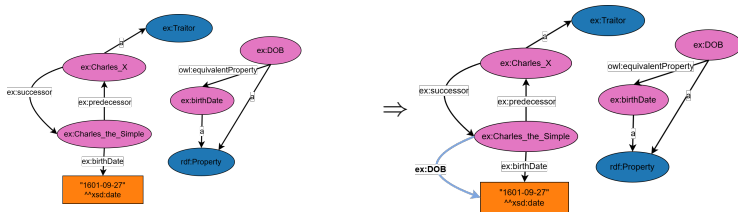
IF `ex:Napoleon owl:sameAs ex:Napoleon_I` AND `ex:Napoleon_I owl:sameAs ex:The_Eagle` THEN `ex:Napoleon owl:sameAs ex:The_Eagle`.

■ E4 (Equivalent Object) Example:

IF `ex:Napoleon owl:sameAs ex:Napoleon_I` AND `ex:Louis_X ex:ancestorOf ex:Napoleon` THEN `ex:Louis_X ex:ancestorOf ex:Napoleon_I`.

IF `ex:Napoleon owl:sameAs ex:The_Eagle` AND `ex:Louis_X ex:ancestorOf ex:Napoleon` THEN `ex:Louis_X ex:ancestorOf ex:The_Eagle`.

Example: OWL Axioms Group G (2/2)



Axiom:

■ G3 (Equivalent Property Instances) Example:

IF `ex:birthDate owl:equivalentProperty ex:DOB` AND `ex:Charles_the_Simple ex:birthDate "1601-09-27"^^xsd:date` THEN `ex:Charles_the_Simple ex:DOB "1601-09-27"^^xsd:date`.

Examples of SPARQL Queries (1/6)

The following SPARQL queries can be executed over https://labs.tib.eu/sdm/clarify_mappings_and_ontology/sparql

Query 1: Count how many data type properties each class has in the ontology.

```
PREFIX rdfs: <http://www.w3.org/2000/01/rdf-schema#>
PREFIX rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#>
PREFIX owl: <http://www.w3.org/2002/07/owl#>
PREFIX xsd: <http://www.w3.org/2001/XMLSchema#>
SELECT ?class (COUNT(DISTINCT ?dataTypeProperty) as ?numDT)
WHERE {
    ?class a owl:Class .
    OPTIONAL {
        ?dataTypeProperty a owl:DatatypeProperty.
        ?dataTypeProperty rdfs:domain ?class.}}
GROUP BY ?class
ORDER BY DESC(?numDT)
```

Examples of SPARQL Queries (2/6)

Query 2: Count how many object properties each class has in the ontology.

```
PREFIX rdfs: <http://www.w3.org/2000/01/rdf-schema#>
PREFIX rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#>
PREFIX owl: <http://www.w3.org/2002/07/owl#>
PREFIX xsd: <http://www.w3.org/2001/XMLSchema#>
SELECT ?class (COUNT(DISTINCT ?objectProperty) as ?num0)
WHERE {
    ?class a owl:Class .
    OPTIONAL {
        ?objectProperty a owl:ObjectProperty.
        ?objectProperty rdfs:domain ?class. }}
GROUP BY ?class
ORDER BY DESC(?num0)
```

Examples of SPARQL Queries (3/6)

Query 3: Count the number of object properties with the same range class.

```
PREFIX rdfs: <http://www.w3.org/2000/01/rdf-schema#>
PREFIX rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#>
PREFIX owl: <http://www.w3.org/2002/07/owl#>
PREFIX xsd: <http://www.w3.org/2001/XMLSchema#>
SELECT ?class (COUNT(DISTINCT ?objectProperty) as ?num0)
WHERE {
    ?class a owl:Class .
    ?objectProperty a owl:ObjectProperty.
    ?objectProperty rdfs:range ?class.}
GROUP BY ?class
ORDER BY DESC(?num0)
```

Examples of SPARQL Queries (4/6)

Query 4: List the class(es) that have at least two data type properties.

```
PREFIX rdfs: <http://www.w3.org/2000/01/rdf-schema#>
PREFIX rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#>
PREFIX owl: <http://www.w3.org/2002/07/owl#>
PREFIX xsd: <http://www.w3.org/2001/XMLSchema#>
SELECT DISTINCT ?class
WHERE {
    ?class a owl:Class .
    ?dataProperty1 a owl:DatatypeProperty.
    ?dataProperty1 rdfs:domain ?class.
    ?dataProperty2 a owl:DatatypeProperty.
    ?dataProperty2 rdfs:domain ?class.
    FILTER (?dataProperty1 != ?dataProperty2)}
```

Examples of SPARQL Queries (5/6)

Query 5: List the class(es) that have at least two object properties.

```
PREFIX rdfs: <http://www.w3.org/2000/01/rdf-schema#>
PREFIX rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#>
PREFIX owl: <http://www.w3.org/2002/07/owl#>
PREFIX xsd: <http://www.w3.org/2001/XMLSchema#>
SELECT DISTINCT ?class
WHERE {
    ?class a owl:Class .
    ?objectProperty1 a owl:ObjectProperty.
    ?objectProperty1 rdfs:domain ?class.
    ?objectProperty2 a owl:ObjectProperty.
    ?objectProperty2 rdfs:domain ?class.
    FILTER (?objectProperty1 != ?objectProperty2)}
```


Examples of SPARQL Queries (6/6)

Query 6: List the class(es) that have data types and object properties.

```
PREFIX rdfs: <http://www.w3.org/2000/01/rdf-schema#>
PREFIX rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#>
PREFIX owl: <http://www.w3.org/2002/07/owl#>
PREFIX xsd: <http://www.w3.org/2001/XMLSchema#>
SELECT DISTINCT ?class
WHERE {
    ?class          a owl:Class .
    ?objectProperty a owl:ObjectProperty.
    ?objectProperty rdfs:domain ?class.
    ?datatypeProperty a owl:DatatypeProperty.
    ?datatypeProperty rdfs:domain ?class.}
```

Comparison of RDFS and OWL

RDFS	OWL
Data model to represent RDF schemas in terms of classes, properties (object and data type properties, and domain and range)	Logical Equivalence can be expressed
Cardinality constraints cannot be represented	Cardinality constraints can be represented
RDFS Axioms are used to deduce missing relationships	OWL Axioms are used to deduce missing relationships and constraints

References I



D. Gries and F. B. Schneider.

A logical approach to discrete math.

Springer, 1994.